

# RAPPORT INDIVIDUEL SAE Web/BD

- Ilane Riotte – info22 -

## SYNTHÈSE :

Objectif de cette SAE : Développer une application complète de gestion de vols et d'aéroports (API Flask, Web SPA, Mobile Flutter). Je me suis principalement occupé du développement complet de l'application Mobile (Flutter), de l'architecture logicielle (MVVM/Repository) et de la partie Base de Données, notamment sur la mise en place de Triggers Oracle pour garantir l'intégrité des données lors des insertions de vols.

## ANALYSE DE CODE :

Passage sélectionné : Logique de filtrage multi-critères dans vols\_screen.dart.

```
void _filtrerVols(String motCle) {
  setState(() {
    if (motCle.isEmpty) {
      _volsFiltres = _tousLesVols;
    } else {
      _volsFiltres = _tousLesVols.where((vol) {
        final recherche = motCle.toLowerCase();
        // Accès propre via les propriétés de l'objet Vol
        return vol.compagnie.toLowerCase().contains(recherche) ||
               vol.depart.toString().contains(recherche) ||
               vol.arrivee.toString().contains(recherche) ||
               vol.numVol.toString().contains(recherche);
      }).toList();
    }
  });
}
```

## Difficultés identifiées :

- Gestion des types : Conversion des int (num\_vol, IDs) en String pour permettre une recherche textuelle fluide sans crash.
- Réactivité de l'UI : Assurer que le ListView se mette à jour instantanément via setState sans saccades lors de la saisie.
- Performance : Filtrage côté client pour éviter des appels API répétés à chaque caractère tapé.

## Solutions appliquées :

- Utilisation systématique de `.toString()` et `.toLowerCase()` pour uniformiser la comparaison.
- Utilisation de la méthode `.where()` de Dart pour un filtrage efficace de la liste d'objets Vol.
- Mise en place de l'architecture Repository pour séparer la donnée brute de la logique d'affichage.

## **CONTRIBUTION BD:**

En complément du développement mobile, j'ai travaillé sur la couche SQL pour optimiser les interactions.

- Triggers de sécurité : Création d'un trigger BEFORE INSERT sur la table VOL pour vérifier que l'aéroport de départ est différent de l'aéroport d'arrivée avant l'enregistrement.
- Vues simplifiées : Conception d'une vue SQL joignant VOL et AEROPORT pour faciliter l'affichage des noms de villes dans les futurs développements, réduisant la complexité des requêtes pour les clients Web et Mobile.

## **DÉMONSTRATION DE COMPÉTENCES :**

### **RÉALISER**

AC21.02 : Ergonomie mobile (Navigation par onglets, interface fluide, thématique cohérente).

AC21.03 : Application des bonnes pratiques de développement (Clean Code, architecture en dossiers, typage fort).

### **OPTIMISER**

AC22.02 : Filtrage de données côté client pour réduire la charge serveur.

AC22.04 : Optimisation des requêtes SQL (Indexation des IDs d'aéroports).

### **ADMINISTRER**

AC23.01 : Mise en place d'une application mobile communicante avec une API REST.

AC23.02 : Gestion de l'environnement de développement Flutter (SDK Dart, simulateur/Chrome).

### **GÉRER**

AC24.02 : Sécurité des données (Validation des entrées utilisateur, SQL paramétré côté API).

AC24.03 : Restitution visuelle de données complexes (JSON vers Objets Dart, rendu en liste Card).

### **COLLABORER**

AC26.02 : Utilisation rigoureuse de Git (gestion des branches, résolution de conflits de fusion).

AC26.04 : Documentation du code et rédaction de rapports de suivi.